

Machine Learning Model for Diabetes Prediction Using Parallel Computing: Comparative Study

Malak Aljabri¹, Shahd Alsaleh², Maha Alsayed², Ghadi Alkehily², Taif Alahmadi², and Lina Alrefi²

¹*Department of Computer and Network Engineering, College of Computing Umm Al-Qura University
Makkah 21955, Saudi Arabia*

²*Department of Computer Science and Artificial Intelligence, College of Computing, Umm Al-Qura University
Makkah 21955, Saudi Arabia*

Abstract — Since diabetes affects how the body uses glucose, a vital source of energy, and can harm the blood as well as organs like the kidneys, heart, eyes, and other tissues, it has become one of the most prevalent and difficult health issues in the world. So, it is necessary to develop a system that can accurately predict diabetes patients using medical clues and artificial intelligence (AI)-based techniques such as Machine Learning (ML) have proven to be effective in predicting diabetes. The use of sequential machine learning methods can be difficult due to their time-consuming nature and limited efficiency in real-time predictions. Additionally, these approaches may struggle to handle large amounts of data. To construct accurate, reliable, and efficient models for diabetes detection within a limited amount of computing time, the integration of parallel computing techniques in machine learning appears to be a practical solution. In this study, we utilized parallel techniques in Python to train ML models and provided a comparative study for different parallel techniques. We used the Pima Indian Diabetes (PIDD) dataset with 506 records for training and 218 for testing. Five experiments were conducted to compare the performance of different execution techniques. The first experiment was based on sequential execution, while the next three experiments used parallel execution techniques. These techniques included threading backend with 4 cores in Python parallel, multiprocessing backend with 4 cores in Python parallel, and loky backend with 4 cores in Python parallel. We deployed two models, namely Decision Tree (DT) and Linear Regression (LR), to conduct the experiments. Our results showed that LR with multiprocessing technique achieved a higher accuracy of 78% and faster with a speedup of 39. Lastly, the results in general showed that parallel execution is faster and more efficient than sequential execution.

Keywords— parallel processing; Diabetes; machine learning; Decision Tree; Linear Regression

I. INTRODUCTION

Diabetes affects 422 million people worldwide and is projected to increase to 490 billion by 2030, according to the World Health Organization [1]. Diabetes is a major chronic disease whose prevalence is continuously increasing. Among young people, diabetes affects how your body handles glucose (sugar), an important source of energy. Diabetes is classified into two types, which are type 1 diabetes and type 2 diabetes. Type 1 diabetes is an autoimmune condition in which the immune system mistakenly attacks and destroys insulin-producing beta cells in the pancreas. Type 2 diabetes is basically insulin resistance, where the body's cells can't process insulin properly, and it can affect children and adolescents. Over time, the pancreas does not produce enough insulin. Lifestyle factors, genetics, and obesity are common risk factors. Untreated and undetected diabetes can cause blood sugar levels to fluctuate and, in extreme cases, damage organs such as the kidneys and eyes. Early and precise diagnosis of diabetes mellitus is a significant challenge for healthcare professionals, especially in its initial stages.[1]

The healthcare sector manages extensive databases, including structured, semi-structured, and unstructured data. These databases provide fertile ground for the application of big data analytics. By using machine learning techniques to analyze these healthcare databases, it becomes possible to develop AI models capable of detecting diabetes and predicting it. These models have the potential to greatly enhance early diagnosis, personalized treatment plans, and improve patient care.

Artificial intelligence techniques such as machine learning have been increasingly utilized in the medical sector filed and offered valuable support by providing a reference point for

gaining preliminary insights into different disease in terms of prediction, classification, or detections which greatly contributed to reducing the workload in healthcare. A considerable amount of research has been dedicated to automating diabetes prediction through machine learning and ensemble techniques, often utilizing different open-source datasets for analysis. Predicting diabetes early and with accurate results and can save many human lives. The purpose of this investigation is to assess the classifiers that can predict the probability of disease in patients with the greatest precision and accuracy by using Machine Learning and parallel computing.

Unfortunately, ML models need a lot of computational power, which results in a longer computation time. Usually, in single processor environments, the ML algorithms cause a significant delay in model processing, from training to classification due to the bottlenecks. Sequential ML computing is often inappropriate for handling huge datasets. On the other hand, parallel computation offers exceptional opportunities to implement these large-scale problems. In our study, the main purpose is not to increase accuracy but the model prediction performance by applying different parallel computing techniques for the machine learning models and conduct comparative evaluation study. We studied the three parallel ML techniques in python namely: multithreading, multiprocessing, and loky which can be effectively deployed to reduce the execution time for the ML algorithms. Those three techniques were studied on Linear Regression (LR) and Decision Tree (DT) machine learning algorithms.

The key contributions of this paper can be summarized as follows:

- Conducting comparative performance evaluation for the sequential and three different parallel python techniques (multithreading, multiprocessing, and loky) utilizing four processing cores on two ML models (LR and DT) to enhance the detection of Diabetes.
- Making a significant contribution to AI, healthcare, and parallel computing literature, especially with limited studies available for comparing the performance of parallelism techniques on ML in this domain.

This paper is structured as follows: Section 2 clarifies the previous studies that deployed sequential and parallel ML techniques. Section 3 presents the methodology followed to carry out the experiments. Section 4 discusses the experimental setup and results obtained. Section 5 consolidates the paper by providing a conclusion and future work.

II. LITERATURE REVIEW

Diabetes is a long-term medical condition characterized by persistently high blood sugar levels brought on by inadequate insulin production, inadequate insulin

utilization, or both. The use of predictive machine learning algorithms in the context of diabetes mellitus is explored in this section. However, there is still room for improvement in the Diabetes prediction. The objective of this study is to develop machine learning algorithms in parallel. It also aims to highlight the significance of early detection to reduce complications and to clarify the critical role that predictive machine learning algorithms play in the management of diabetes. Several articles were pulled from Google Scholar. Ranging from between 2023 and 2018.

A study aimed to identify the most effective machine-learning model for diabetes prediction by Kangra et al. [2] They compared different algorithms including Naïve Bayes (NB), Random Forest (RF), Support Vector Machine (SVM), K-Nearest Neighbor (KNN), Decision Tree (DT), and Logistic Regression (LR), to analysis performance indices and error rates. The used dataset is The German dataset and Pima Indian Diabetic (PID) dataset from Kaggle, with analysis conducted in WEKA 3.8.6 software using python. The PID dataset comprised 9 attributes and 768 instances, with the goal of diabetes classification. The German dataset also had nine variables. In both cases, the primary objective was diabetic status determination. SVM performed well with a 74% accuracy for the PID dataset, while KNN and RF excelled, achieving an impressive 98.7% accuracy for the German dataset. The future work that the researchers suggested could involve exploring hybrid models and assess their performance alongside other algorithms, providing valuable insights for further research.

A study by Sivaranjani S et al. [3] who aimed to predict diabetes using machine learning algorithm, which is support SVM and RF. The dataset has been used is PIMA, they addressed missing values by preprocessing, and performed feature selection using the “wrapper method” to Increases the efficiency of the model and reduces its complexity. Special analytical tools Principal Component Analysis (PCA) is used for dimensionality reduction, optimizing data set complexity Effective modeling. The authors used python programming language. The results showed that feature selection affected RF and SVM classifier accuracy, the proposed model achieved RF with 83% accuracy and 81.4% SVM, which outperforms the algorithms. Future research may refine the features selection and explore new dimensionality reduction techniques to move forward increase the efficiency of the model.

A study by Soni M, et al. [4] suggested using various classification machine learning methods for early diabetes prediction, which are KNN, LR, DT, SVM, Gradient Boosting (GB), and RF. All these classifications and ensemble methods were applied to detect which of them gave the highest accuracy for diabetes prediction. The PIMA was used to gather the data and underwent preprocessing in two steps, which were Missing Values removal and splitting of

data, utilized in all the algorithms implemented using Python. While all these algorithms demonstrated accuracy exceeding 65%, RF stood out with the highest accuracy among them at 78% , making it the more accurate algorithm in predicting diabetes. This paper didn't include a discussion section addressing the practical implications of the research and potential future directions, which was important and useful to include .

a study by Sarwar Muhammad et al.[5] proposed the deployment of machine learning algorithms to predict diabetes and help healthcare professionals make timely decisions on the health and treatment of the patient. The study six different algorithms which are SVM, DT, LR, RF, KNN, and NB, the dataset is the Pima Indians Diabetes. The dataset was divided into two sections training data (70%) and data (30%). The authors used Python programming language In the Enthought Canopy tool. The results showed that SVM and KNN achieved the highest accuracy of 77% compared to other algorithms tested. In order to enhance the accuracy of the study it's better to integrate other methods into the model and test it with a larger database.

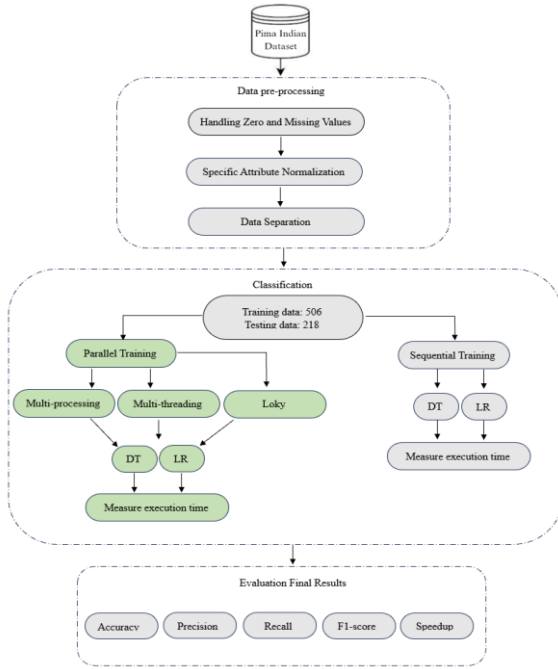
In a study by Rani K [6] that aimed to develop a highly accurate system for early diabetes prediction by combining various machine learning methods. The study was used the John Diabetes Database, which had nine features, with 2000 data points. The study used John's diabetes database, which contains nine attributes, with 2,000 data points. The study compared the accuracy of training and testing diabetes prediction when implemented by Python on several classifications methods, including KNN, LR, DT, SVM, and RF. In the study, KNN and LR achieved an accuracy of up to 78%, while (DT) on all previous algorithms with an accuracy of 99%, and the most influential attribute was “glucose”. RF reported an accuracy of 94%, with “glucose” and “BMI” as key attributes. SVM delayed accuracy by 76%. In the future it is possible to expand the system to predict or diagnose other diseases and explore additional machine learning algorithms to improve accuracy.

Year	Algorithm	Dataset	Performance measures
2023	NB, RF, SVM,KNN,DT and LR	Germany and Pima Indian diabetic (PID) diabetes datasets	Identifying the accuracy
2021	SVM and (RF).	The PIMA Indians Diabetes (PID)	The performance measures mentioned in the paper are test accuracy, validation accuracy, sensitivity, and specificity
2020	KNN, LR,DT,RF ,GP and SVM	The PIMA Indians Diabetes (PID)	Identifying the accuracy
2020	KNN, LR,DT,RF and SVM	John Diabetes Database	Identifying the accuracy
2018	SVM, DT, LR, RF, KNN, and NB	The PIMA Indians Diabetes (PID)	Identifying the accuracy

Table 1: Literature Review

III. METHODOLOGY

We trained the models in parallel in Python using various multiprocessing and multithreading strategies. Then, we measured the training time in sequential execution and compared it to the parallel execution time, with a publicly accessible dataset, which distinguish diabetes from other medical conditions. An overview of the initial methodological steps taken is shown in Figure 1 below, and these will be covered in more detail in the ensuing subsections.



Figuer1: The methodology adopted.

A. Dataset Description.

The dataset used was developed by the National Institute of Diabetes and Digestive and Kidney Diseases, and its primary purpose is to predict the likelihood of a patient developing diabetes based on specific diagnostic tests. This dataset has been updated biennially since 1965.[6]

Tables 2 and 3, show the diabetic dataset characteristics, data type, and statistics. The classifier predicts whether the patient has diabetes based on the data set characteristic, which is intended for binary classification, with class value 1 showing persons who have gotten a “diagnosis of diabetes” and class 0 standing for those who have not obtained a diabetes diagnosis.

PIDD	Dataset
768	Samples
8	I/P Attributes
2	O/P Classes
9	Total Attributes
Nil	Missing values
Nil	Noisy-Attributes

Table 2: Data Set Statistics

There are 768 observations in the data set, five hundred for training and 268 for testing, with no missing values. It is also worth noting that the dataset holds impractical values like zero for body mass index and insulin. There are a total of nine qualities, eight of which are independent, and one is reliant. All the qualities are either discrete or continuous.

Sl. No	Attribute	Description
1	No. of times Pregnant	Discrete data of type int64
2	Plasma Glucose Concentration	Discrete data of type int64
3	Diastolic Blood Pressure (mm Hg)	Discrete data of type int64
4	Skin Thickness (mm)	Discrete data of type int64
5	Insulin mu U/ml	Discrete data of type int64
6	BMI (Weight/ Height) (kg/m2)	Continuous data of type int64
7	Diabetes Pedigree Function	Continuous data of type int64
8	Age	Discrete data of type int64
9	Outcome Class	Discrete data of type int64

Table 3: Attribute Description

B. Data Preprocessing

We noticed biologically implausible zero values in the PIMA dataset, so we employed a hybrid approach to manage zero values, eliminating missing values from the parameters that have the greatest effect on the outcome and normalizing the remaining parameters using the median, to create efficient models capable of accurately detecting cases of diabetes and non-diabetes.

We normalized Skin Thickness and Insulin attributes by filling null values by median and then eliminating zero values by median because blood pressure and glucose are critical for deciding diabetes and their null fraction is extremely small, so it should be better to remove invalid entries.

presently we've got clean data, we can separate it into two datasets: training 70% and testing 30%, using the sklearn train split technique.[7]

C. Classification Phase

We assessed two ML, which are Decision tree (DT) and Linear Regression (LR), to train and test the dataset for predicting diabetes. Both sequential and parallel training methods were used for all the models. In sequential training, only one execution thread was used to train the models; in parallel training, execution 4 threads were used to train the models. We will go into more detail in this section about the three principal areas of our study: sequential training, parallel training, and evaluation phase.

- 1) Sequential Training: The Decision Tree and Linear regression are models sequentially trained to gather performance metrics and keep track of execution time. The following subsections supply a brief overview of the Decision Tree and Linear regression implementations.
- 2) Decision Tree: Decision tree supervised learning algorithm used to solve classification and regression problems. It is usually preferred in such cases. The classifier has a tree-like structure, where the internal nodes represent the dataset's features, the branches decide the decision-making process, and each leaf node stands for the classification result.[8]
- 3) Linear regression: Linear regression is a type of supervised machine learning algorithm that decides how dependent a variable is, and how one or more independent features are related linearly. The algorithm seeks out the best linear equation that can predict the value of the dependent variable based on the independent variables.[9]
- 4) Parallel Training: In the following subsections, the techniques adopted to convert the sequential ML models that is, DT and LR to be trained in parallel are described. This section's main goal is to examine, analyze, and investigate potential parallel processing techniques for training ML models in Python.

Threading Backend and Number of Cores in Python Parallel

In this technique, the sklearn joblib library was imported into the working environment to use Python's built-in parallel backend threading features. The environment's ML models can be parallel trained by using the commands `import joblib` and `from joblib import parallel_backend`. which enables them to utilize all the machine's available cores, speeding up the training process. Moreover, we added another argument called `n_jobs` that gives us more control over the number of active threads or CPU cores, means that use multi-threading to parallelize the training process and allocate specific number of threads for that purpose. For example, `n_jobs = -1` instructs the computer to use every thread that is available, while `n_jobs = 1` runs the program in a single thread, and so on. Before fitting the model can be written this line of code `"with parallel_backend('threading', n_jobs=#)"` where `#` represents the number of cores or processes to utilize to train the models parallelly.

Multi-Processing Backend and Number of Cores in Python Parallel

This technique makes use of the same code and libraries as those mentioned above. However, it makes use of processes rather than threads. The "multiprocessing" backend utilizes individual processes, making it a suitable choice for parallelism within a single host. However, it's considered a legacy approach. The code for the "multiprocessing" technique is implemented using `"with parallel_backend('multiprocessing', n_jobs=#)"`.

Loky Backend and Number of Cores in Python Parallel

The parallel backend libraries and codes of Python are also used in this technique. The "loky" backend allows for adaptive parallelism and is particularly valuable for tasks that benefit from efficient multi-processing execution. It is based on single-host processes and is primarily based on multi-processing execution. To implement the "loky" technique, use this line `"with parallel_backend('loky', n_jobs=#)"`.

Evaluation Phase

Different performance metrics can be used to compare ML classifiers. In this study, the models that were deployed were evaluated and their performances were compared using accuracy, precision, recall, and F1-score. The confusion matrix, which is used to evaluate the accuracy of the classifier, supplies a comparison of the model's predicted classifications alongside the actual classifications and it consists of four main values. In addition, the execution times for both sequential and parallel programs were given in seconds, and the speedup was calculated using this data.

- True Positive (TP): The model correctly predicts that a person has diabetes, and the person does have diabetes.
- False Positive (FP): The model incorrectly predicts that a person has diabetes when the person does not have diabetes.
- True Negative (TN): The model correctly predicts that a person does not have diabetes, and the person does not have diabetes.
- False Negative (FN): The model incorrectly predicts that a person does not have diabetes when the person does have diabetes.

Accuracy is the percentage of how often a classification ML model is correct overall. By comparing the number of accurate predictions to the total number of instances in a dataset and calculated using the following formula :

$$Accuracy = (TP + TN) / (TP + FP + TN + FN)$$

Precision refers to the number of true positives divided by the total number of positive predictions and is calculated using the following formula:

$$Precision = TP / (TP + FP)$$

Recall is the proportion of correctly predicted classes to all positive classes and is calculated using the following formula:

$$Recall = TP / (TP + FN)$$

F1-score supply a single metric that balances the trade-off between precision and recall, and is using the following formula:

$$F1 - score = (2 \times Precision \times Recall) / (Precision + Recall)$$

Speedup is the proportion of the time it takes to complete a task sequentially to the time it takes to complete the same task in parallel calculated by following formula:

$$Speedup = (SequentialExecutionTime) / (ParallelExecutionTime)$$

IV. RESULT AND DISCUSSION

A. Experimental setup

To conduct the experiments, we used Python 3.11.5 on the Visual studio code platform. The computer that has been used runs Windows 11 operating system. And the device

processor was an AMD Ryzen7 5800H with 16 logical processors and eight cores with RAM 16 GB. The Pima Indian dataset selected features is nine attributes as follows: number of pregnancies, plasma glucose concentration, diastolic blood pressure, triceps skinfold thickness, 2 hour serum insulin, Body Mass Index (BMI), Diabetes Pedigree Function, age and class variable that indicates whether a patient has diabetes or not as shown in Table ???. After data processing the total amount of instances become 724, form this data 506 used for training and 218 for testing. Overall, DT and LR were used in five experiments. Both sequential and parallel computing were included in each of them. The parallel computing was based on threading, multiprocessing and loky

Result

By applying different parallel techniques that remark on the methodology section, the accuracy, precision, recall, F1-score, execution time, and speedup that measured for each parallel technique and sequentially. The results of the evaluation metrics were stable, and no changes were observed while performing our experiments, which show up that our experiments did not suffer any trade-off between speed and performance. The table 4 shows a summary of the results of the evaluation metrics found for each model for two classes, including “true positive” and “true negative.” Both models performed well. However, LR outperformed DT with 78% accuracy while DT had 70% accuracy. The precision results showed that LR has the highest precision in the “true negative” and “true positive” classes at 77% and 83%, respectively, while DT has the lowest precision in the “true negative” and “true positive” classes at 74%,63% respectively. The recall that

measures the model’s ability to accurately identify all related instances of a given class contained by a dataset shows that LR outperformed DT in identifying “true negative” class with 94% while DT was 81%, whereas in identifying “true positive” class, DT was outstripped with 53% and LR with 50%. For F1-score that reflecting the balance between precision and recall indicates that LR had a better score than DT, this conclude that LR was more accurate to predict diabetes than DT.

	DT		LR	
	Class 0	Class 1	Class 0	Class 1
	(true negative)	(true positive)	(true negative)	(true positive)
Precision	74%	63%	77%	83%
Recall	81%	53%	94%	50%
F1-score	77%	57%	85%	62%
Accuracy	70%		78%	

Tabel 4: Evaluation results obtained.

The data begin used shows how applying parallelization technique to LR and DT models minimize execution times. Three parallel approaches employed is Threading, multiprocessing, and multiprocessing with Loki, each of them has its own outcome. The speedup values for linear regression, which range from roughly 3.1 to 3.9, show to us that all parallelization techniques perform better than the sequential approach, with multiprocessing producing the maximum speed up. While the decision tree model also satisfier result from parallelization, the speedup values is lower compared to LR, ranging from about 1.15 to 1.42, with Threading providing the best increase of the speed in this model. The calculated efficiency values, which range from approximately 0.7759 to 0.9853 for Linear Regression and from 0.2884 to 0.3555 for Decision Tree, reveal how effectively the parallelization techniques utilize the available processors with knowing that Values closer to 1 has higher efficiency, that make the efficiency of LR is also better than DT. As a result, the Linear Regression model is more adaptable to parallelization than the Decision tree, especially the Multiprocessing technique. The Decision Tree still performs respectably with Threading. The execution times, speedups and efficiency for various approaches and classifiers are displayed in the table below in table 5, also in Figure 2 and Figure 3.

	Sequ ential	Threading			Multiprocessing			Multiprocessing with Loki		
	Executi on Time in(S)	Execu tion Time in(S)	Spee dup	Effici ency	Execu tion Time in(S)	Spee dup	Effici ency	Execu tion Time in(S)	Spee dup	Effici ency
D T	0.01 1	0.0 08	1.4 22	0.3 56	0.0 09	1.2 38	0.3 09	0.0 10	1.1 53	0.2 88
L R	0.01 3	0.0 04	3.1 04	0.7 759	0.0 03	3.9 412	0.9 853	0.0 040	3.1 82	0.7 96

Table 5: The experiments results summary.

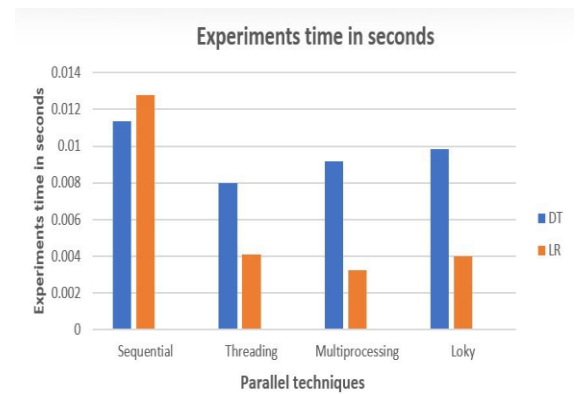


Figure 2: Execution time for both models

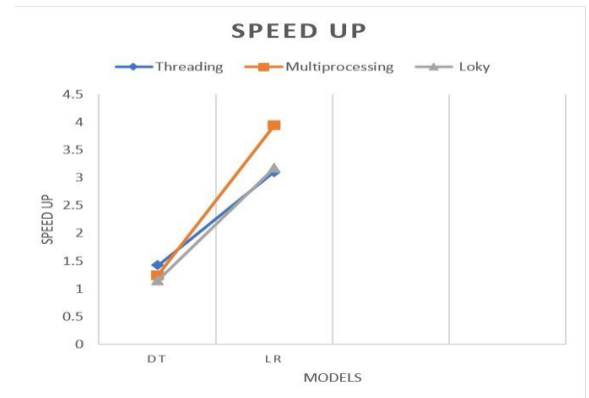


Figure 3: Compare the two models' speed up.

Although the performance of parallel techniques can vary amongst several hardware characteristics. It is important to know that not all models respond positively to a particular technique. In instance, a single technique that produces acceptable results for one model could produce poor results for a different model. Therefore, before putting parallel procedures into practice, it is necessary to carefully study and evaluate their compatibility with various models.

V. CONCLUSION AND FUTURE WORK

In conclusion, Diabetes is a disease that causes many serious complications that greatly affect human health. There is a dire need and quest to predict diabetes early with high performance and less computation time to save many human lives. In this study, we worked on enhancing sequential diabetes prediction and classification by applying parallel processing to speed up the training time taken by ML models. For this purpose, we applied experiments in sequential and parallel using multiprocessing, multi-threading libraries in Python, firstly we applied DT classifier and executed it sequentially, afterward, parallel experiments using the following techniques multiprocessing backend, loky backend, and threading backend. Subsequently, we applied the sequential execution and parallel experiments in the LR classifier. The performance was evaluated and the execution time and speed up were measured for each technique in both models. After conducting the analysis, we thoroughly discussed and compared the results obtained from the sequential and parallel executions. In general, the sequential execution time of the models was reduced using all the parallel processing techniques with a maximum speed up of 3.9 achieved for the LR model using Python multiprocessing backend and the number of jobs set to 4. Whatever, considering the entire performance, multiprocessing using Python parallel backend provided the best results. Therefore, there was no trade-off between performance and speed of execution. The LR achieved the highest accuracy of 78% while DT achieved 70%. This work results contribute to both field of healthcare and parallel processing to ML solutions. For future work, the designed system can be used to predict other diseases, the work can be improved by using a larger dataset, and other machine learning algorithms.

REFERENCES

- [1] "Diabetes." Accessed: Jan. 24, 2024. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/diabetes>
- [2] K. Kangra and J. Singh, "Comparative analysis of predictive machine learning algorithms for diabetes mellitus", *Bulletin of Electrical Engineering and Informatics*, vol. 12, no. 3, pp. 1728–1737, Jun. 2023, doi: 10.11591/EEI.V12I3.4412.
- [3] S. Sivaranjani, S. Ananya, J. Aravinth, and R. Karthika, "Diabetes Prediction using Machine Learning Algorithms with Feature Selection and Dimensionality Reduction", in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2021, vol. 1, pp. 141–146, doi: 10.1109/ICACCS51430.2021.9441935.
- [4] M. Soni and D. S. Varma, "Diabetes Prediction using Machine Learning Techniques", *International Journal of Engineering Research & Technology*, vol. 9, no. 9, Oct. 2020, doi: 10.17577/IJERTV9IS090496.
- [5] M. A. Sarwar, N. Kamal, W. Hamid, and M. A. Shah, "Prediction of Diabetes Using Machine Learning Algorithms in Healthcare", *2018 24th International Conference on Automation and Computing (ICAC)*, pp. 1–6, 2018.
- [6] K. M. J. Rani, "Diabetes Prediction Using Machine Learning", *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp. 294–305, Jul. 2020, doi: 10.32628/CSEIT206463.
- [7] "AnalyzingPima-Indian-Diabetes-dataset". <https://medium.com/analytics-vidhya/analyzing-pima-indian-diabetes-dataset-36d02a8a10e5>.
- [8] "Decision Tree Algorithm in Machine Learning - Javatpoint". <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>.
- [9] "LINEAR REGRESSION IN MACHINE LEARNING - GEEKSFORGEEKS". [HTTPS://WWW.GEEKSFORGEEKS.ORG/ML-LINEAR-REGRESSION/](https://www.geeksforgeeks.org/ml-linear-regression/).